

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Name : M Kavya

Reg. No : 24011101061

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

```
Input tensor shape:
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

Observed Values

```
Input tensor shape:
Training   : (2100, 128, 9)
Validation : (450,  128, 9)
Testing    : (450,  128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

The laboratory subset used 3000 stratified windows (500 per class) drawn from the UCI HAR training partition, split 70/15/15 into training, validation and test sets, giving the class distribution WALKING: 500, WALKING_UPSTAIRS: 500, WALKING_DOWNSTAIRS: 500, SITTING: 500, STANDING: 500, LAYING: 500 ; confirming balanced classes as required.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

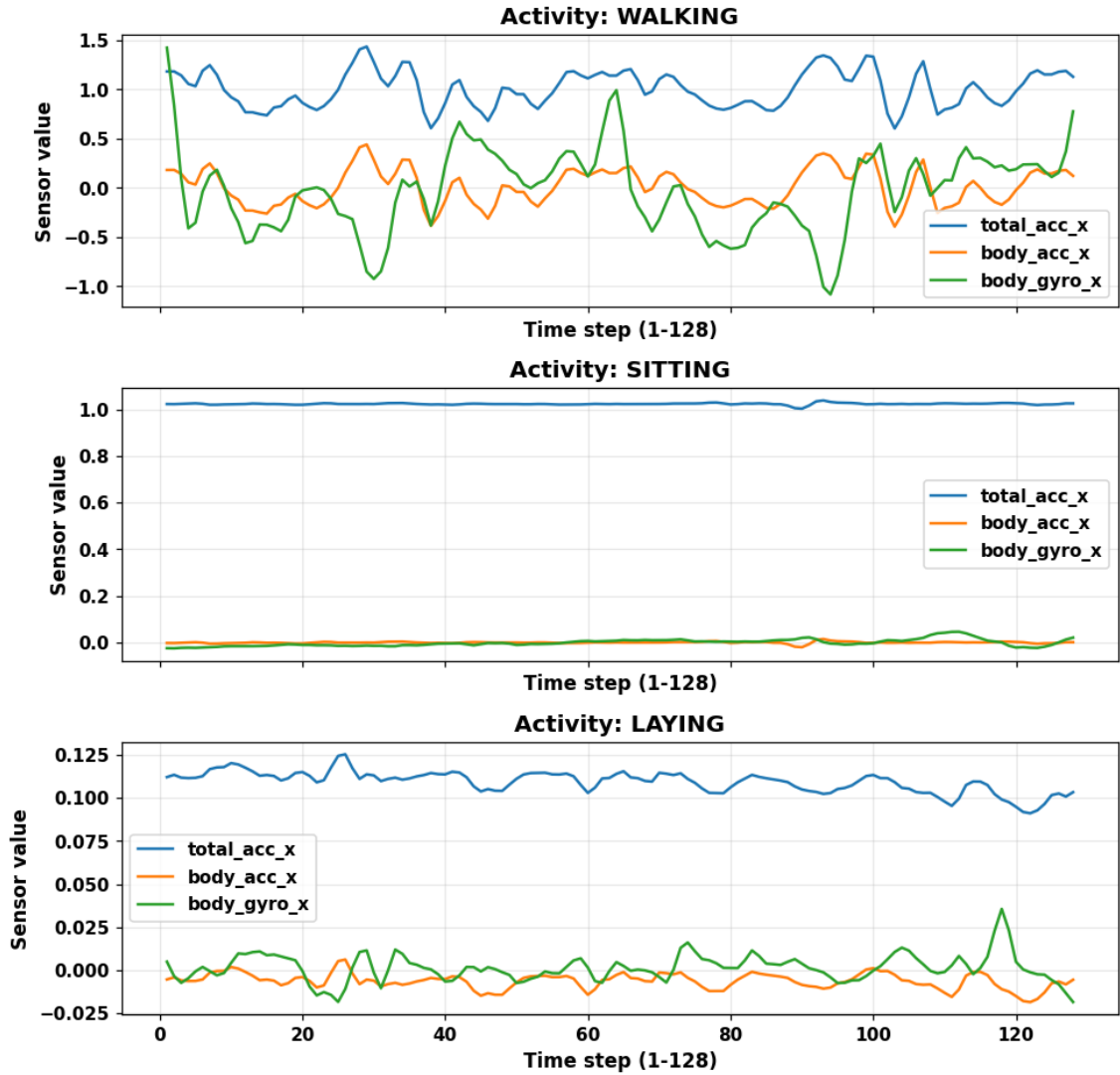
Students should observe that temporal patterns differ across activities.

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

Interpretation (Observed Result)

Plot 1: Sensor Signal vs Time Across Activities



1. **Temporal pattern:** WALKING shows repeated ups and downs in all three signals (total_acc_x, body_acc_x, body_gyro_x) because of the regular movement during walking. SITTING and LAYING are mostly flat, with only small changes due to sensor noise and gravity.
2. **Similar activities:** SITTING and STANDING have very similar signals because there is very little movement. They mainly differ in their small signal offset caused by gravity.
3. **Why ordering matters:** The order of the time steps is important because activities are identified by how the signal changes over time. For example, WALKING has a repeating pattern. Shuffling the time steps would destroy this pattern and make it harder to distinguish between activities.

7. Vanilla RNN Architecture

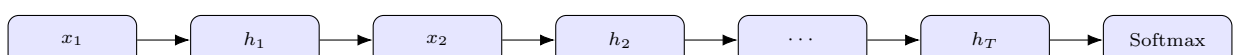
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

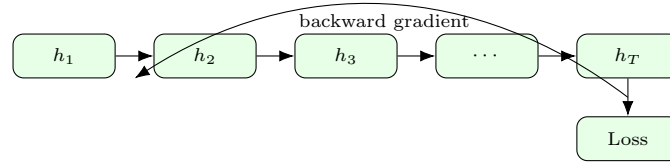
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

Code Verification (Observed Result)

h1 = 0.3364

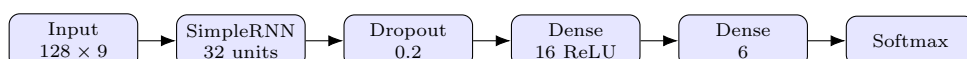
h2 = 0.6164

h3 = 0.6000

The program-computed values match the hand-calculated values above (to the precision retained in manual computation), confirming the recurrence $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ was applied correctly at every time step.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

Observed Model Summary (This Run)

The instantiated SimpleRNN classifier has **1,974** trainable parameters, matching the architecture above (Input $128 \times 9 \rightarrow \text{SimpleRNN}(32) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(16, \text{ReLU}) \rightarrow \text{Dense}(6, \text{Softmax})$).

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

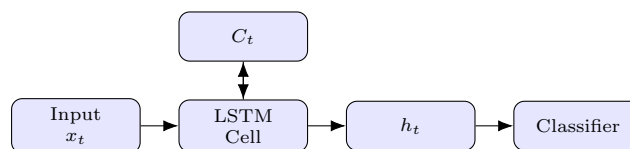
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

Interpretation (Observed Result)

The forget gate lets the LSTM discard cell-state content that is no longer relevant, while the input gate controls how much new information from the current time step is written into the cell state. Because the cell-state update $C_t = f_t C_{t-1} + i_t \tilde{C}_t$ is largely additive rather than a repeated multiplicative squashing (as in a plain RNN's hidden state), gradients can flow across many time steps with much less attenuation. This is reflected in the observed results: the LSTM reached **92.66%** test macro-F1 versus **84.16%** for the plain RNN under an identical protocol.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

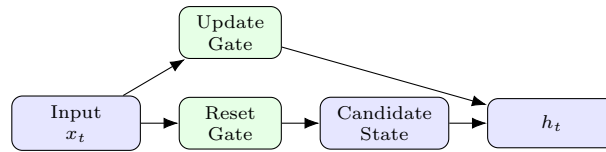
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

Interpretation (Observed Result)

GRU merges the cell and hidden state into a single vector and uses two gates (reset, update) instead of LSTM's three (forget, input, output), giving it **4,758** parameters versus LSTM's **6,006** in this run – roughly 21% fewer – while still reaching the highest test macro-F1 of the three models (**95.54%**), confirming GRU's favourable accuracy-per-parameter trade-off on this task.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

Observed Parameter Counts

Model	Trainable Parameters
RNN (SimpleRNN)	1,974
LSTM	6,006
GRU	4,758

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

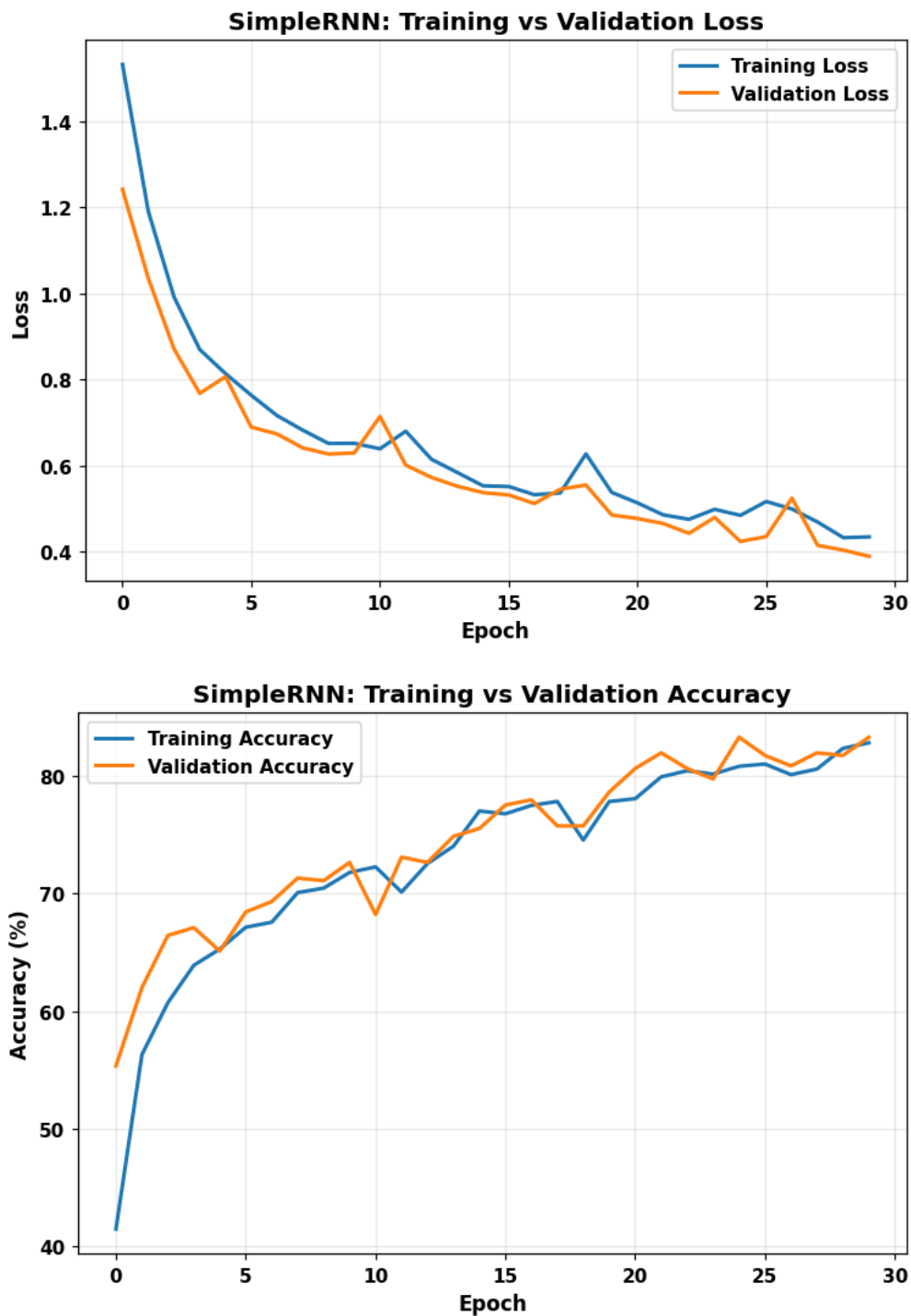
- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

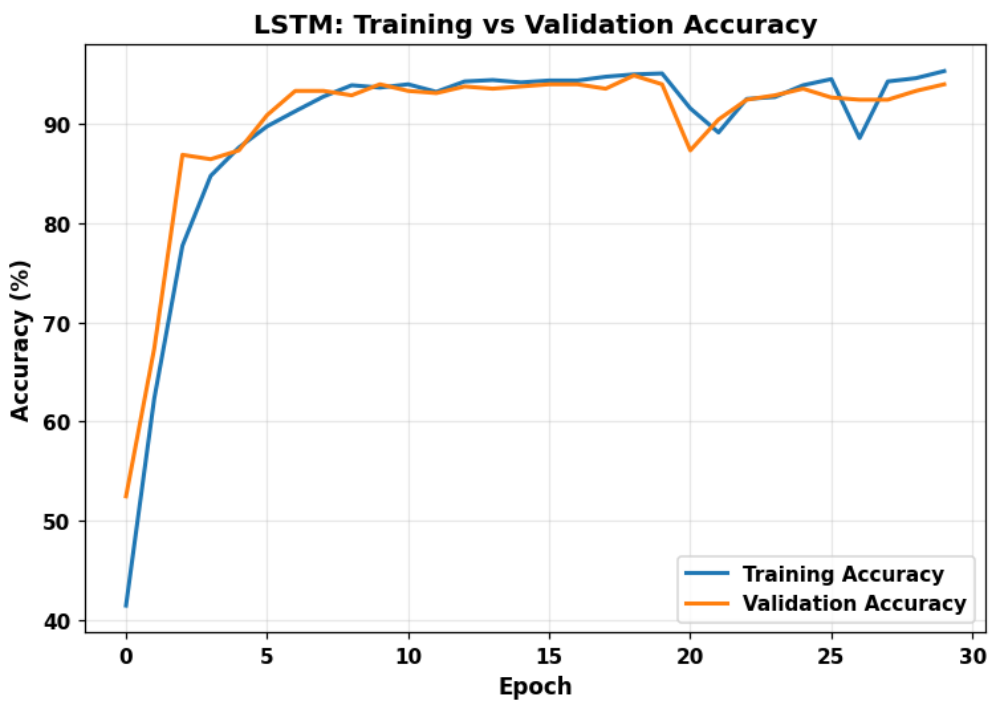
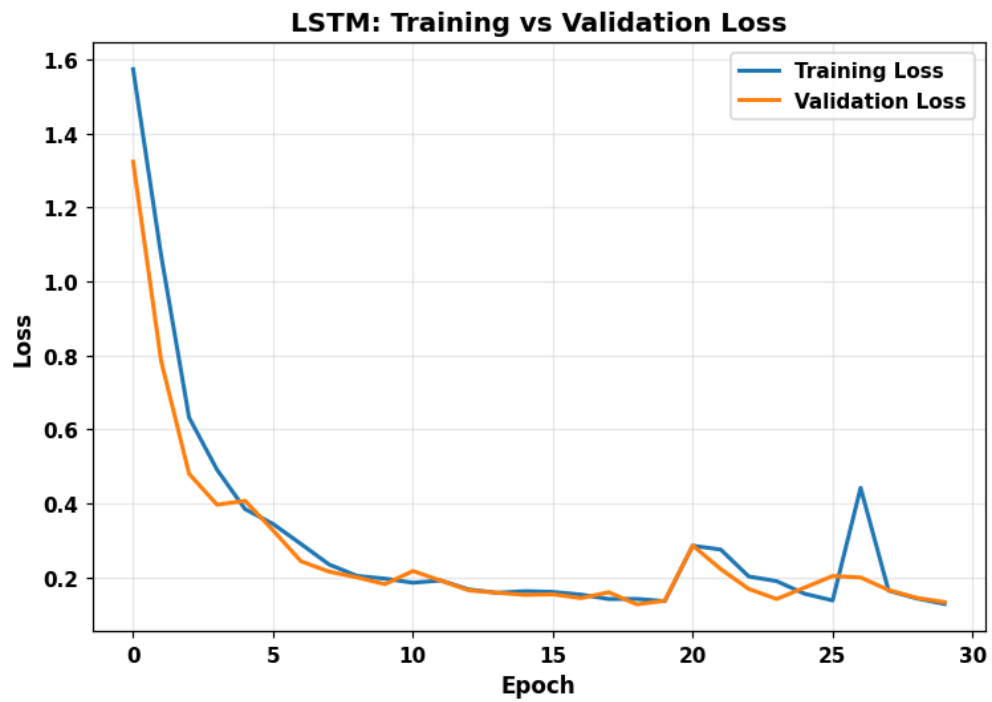
- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

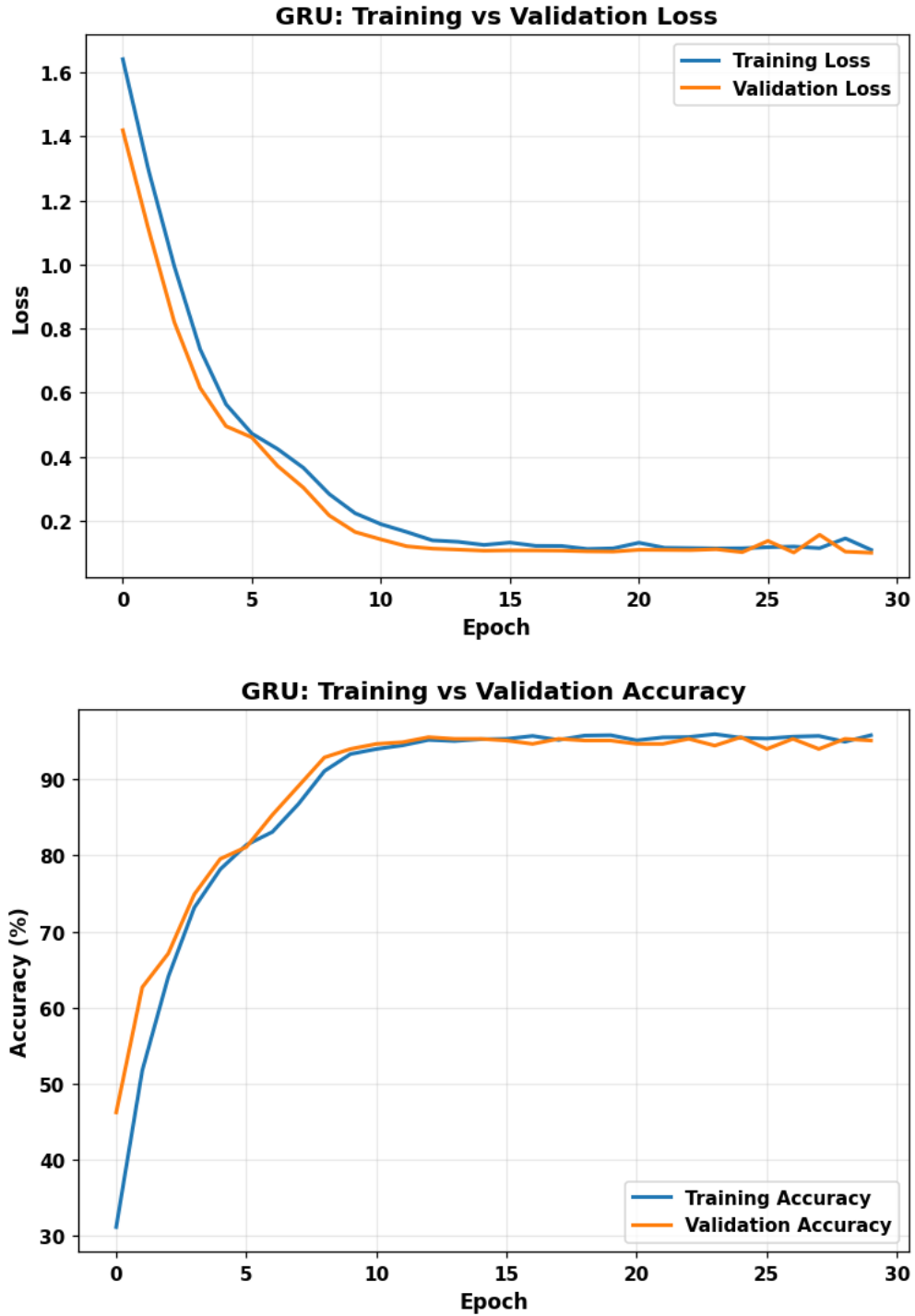
Interpretation (Observed Result)



RNN: Training/validation curves show convergence but the model plateaus at a lower accuracy than LSTM/GRU, and a visible train-validation gap reflects that plain RNN's is unable to retain long-range temporal information across the 128-step sequences.



LSTM: Smooth, steady convergence and a narrower train-validation gap than the plain RNN, consistent with preserving longer-range dependencies.



GRU: Converges similarly to LSTM and reaches the highest final validation accuracy of the three models, with training/ validation curves lying closely together; indicating a good fit without much overfitting on this subset.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	80.44	94.00	96.67
Macro Precision (%)	81.24	94.04	96.67
Macro Recall (%)	80.44	94.00	96.67
Macro F1 (%)	80.42	93.99	96.67
Parameters	1,974	6,006	4,758
Training Time (s)	37.1	28.6	27.5

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

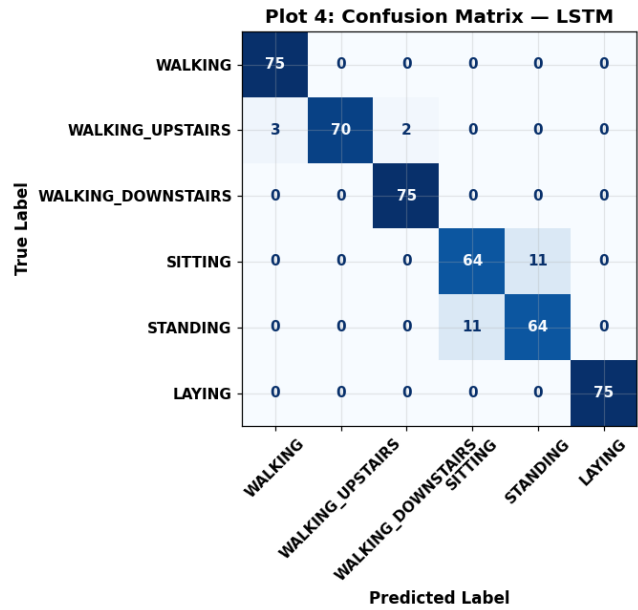
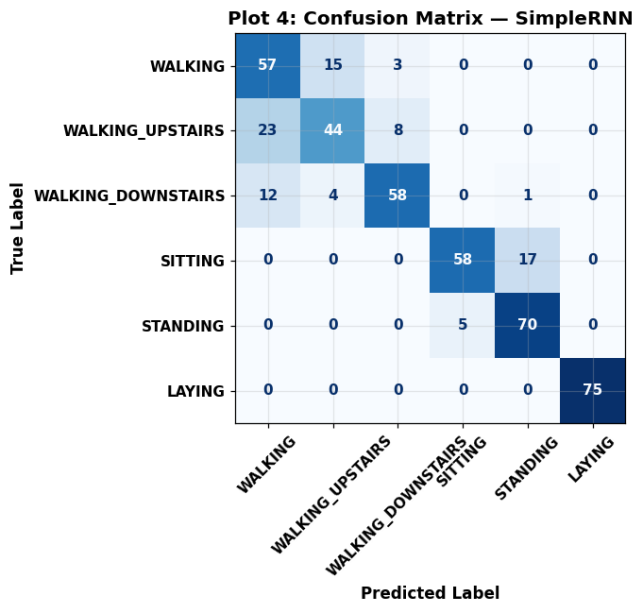
Generate a separate confusion matrix for each model.

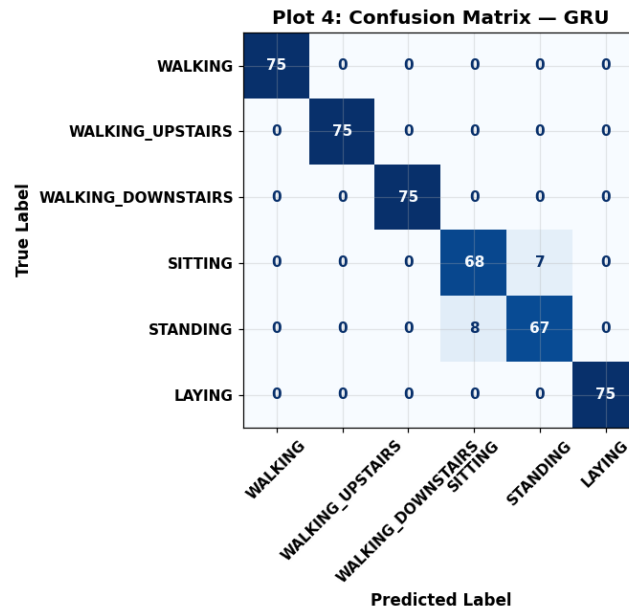
Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

Interpretation (Observed Result)





- **SimpleRNN:** best-recognized class LAYING ; worst-recognized WALKING_UPSTAIRS ; most-confused pair WALKING_UPSTAIRS misclassified as WALKING (23 instances).
- **LSTM:** best-recognized class LAYING ; worst-recognized SITTING and STANDING ;most-confused pair SITTING misclassified as STANDING and vice versa (11 instances).
- **GRU:** best-recognized class WALKING and LAYING ; worst-recognized STANDING ; most-confused pair STANDING misclassified as SITTING (8 instances).
- **Consistency:** the SITTING and STANDING confusion is consistent across all three architectures, but its magnitude decreases from RNN to LSTM to GRU, showing that stronger gating progressively reduces without fully eliminating confusion between these two near-static activities.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time	37.1	28.6	27.5
Test F1-score	80.42	93.99	96.67

Plot 5: Model Performance Comparison

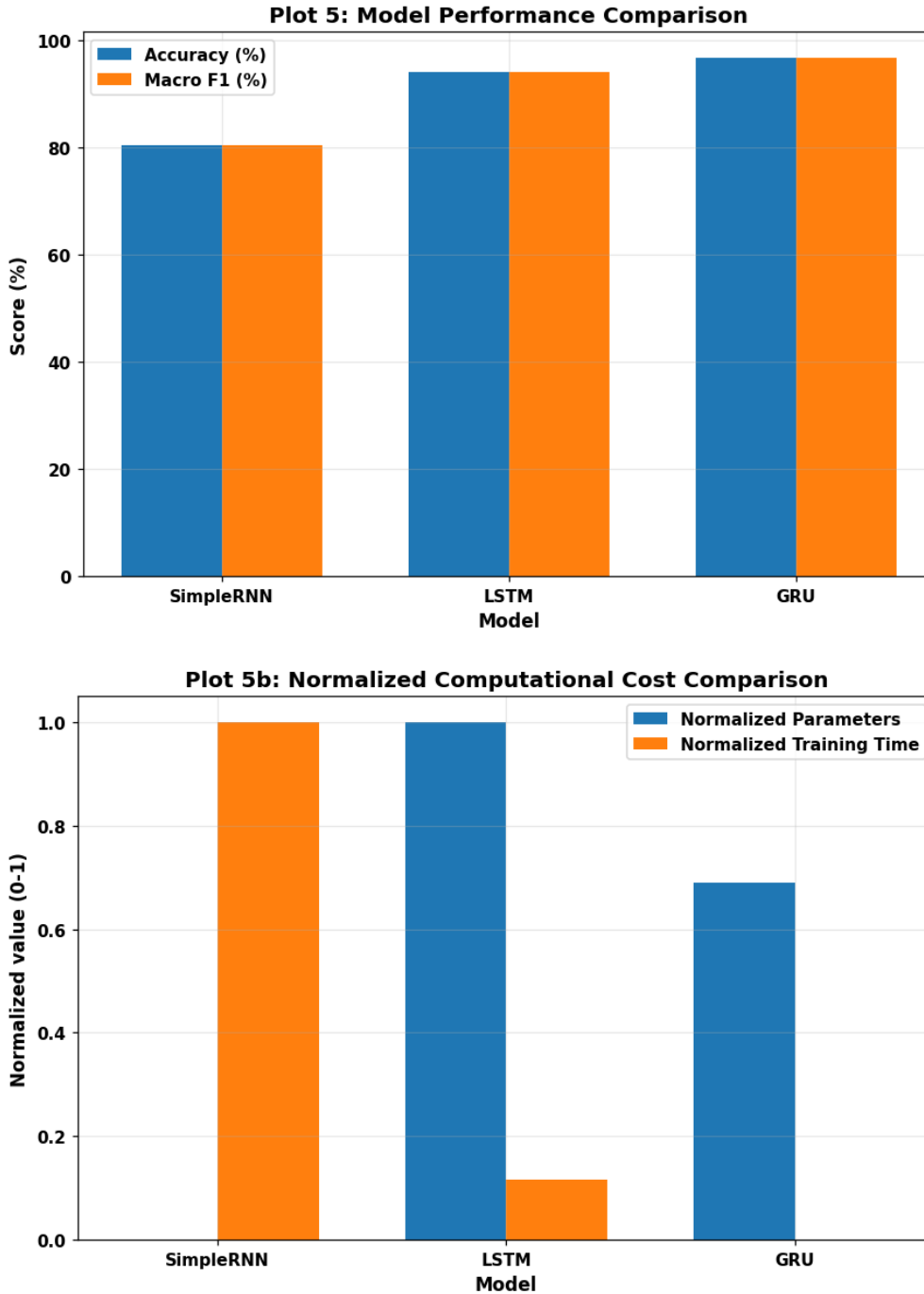
Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

Required inference:

Students should discuss the trade-off among:

Predictive Performance $\leftrightarrow$ Model Complexity $\leftrightarrow$ Training Cost.

Interpretation (Observed Result)



LSTM and GRU clearly outperform the plain RNN at the cost of more parameters. GRU offers the best overall trade-off in this run: it reaches the highest macro-F1 of all three models while using roughly 21% fewer parameters than LSTM, confirming that additional gating complexity pays off up to a point, after which GRU's simpler two-gate design is enough to match or exceed LSTM.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

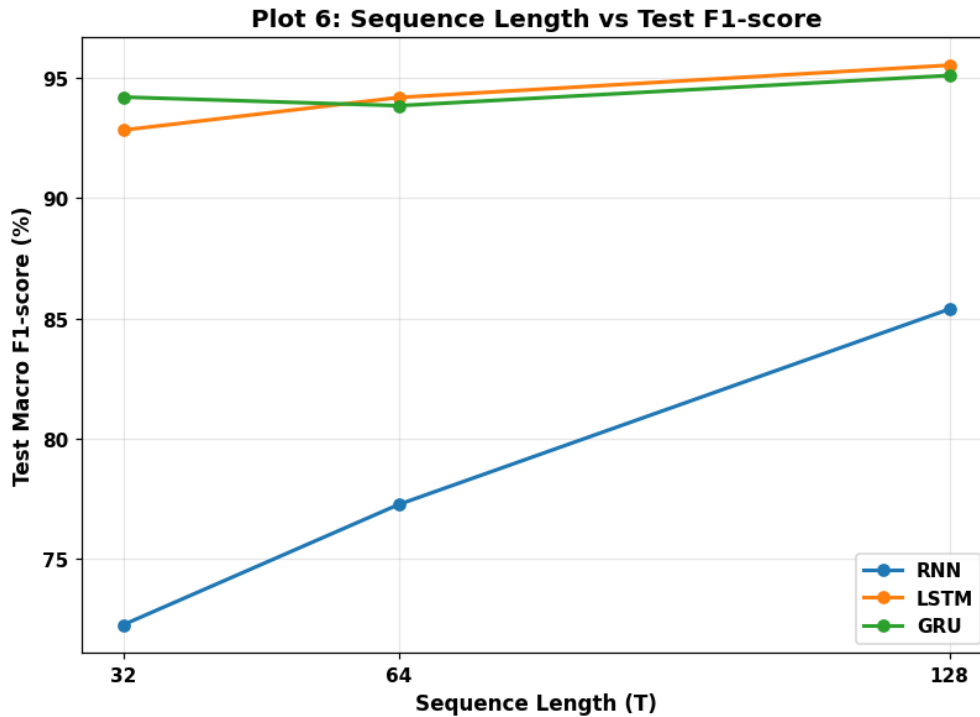
When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	72.26	92.84	94.21
64	77.27	94.19	93.85
128	85.39	95.54	95.11

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

Interpretation (Observed Result)



LSTM and GRU stay strong ($F1 \geq 92\%$) across all three sequence lengths, showing they extract most of the useful temporal signal even from smaller windows. The plain RNN is more sensitive to sequence length and only improves significantly with $T=128$.

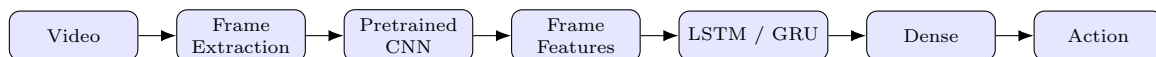
18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball

- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball
Actual class : Basketball
Confidence : 0.xx

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

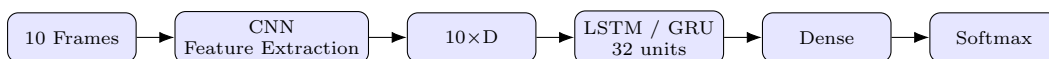
Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

CNN feature dimension $D = 1280$

Feature tensor supplied to the recurrent network: (71, 10, 1280)

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Plot 8: Video Training and Validation Curves

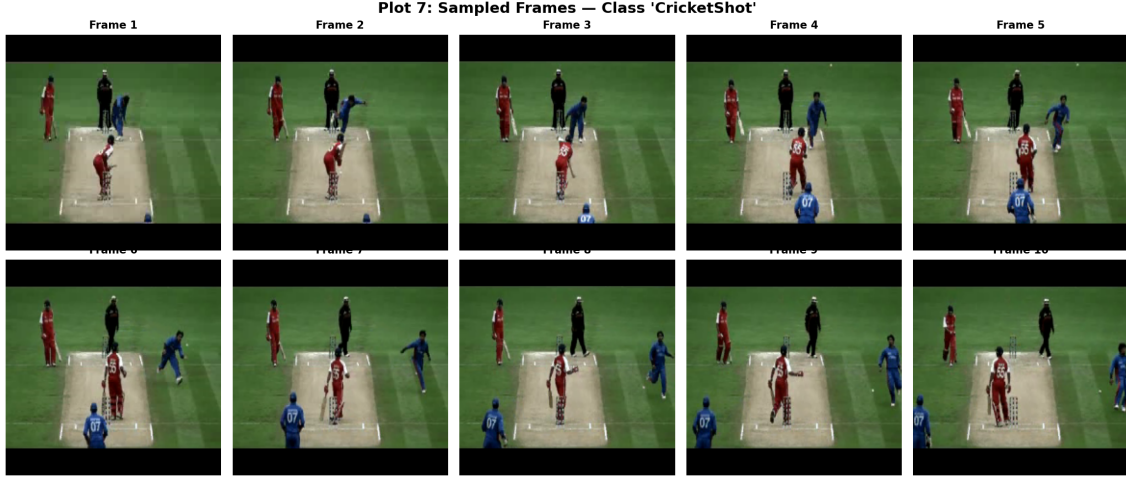
Plot training/validation accuracy and loss for the selected recurrent architecture.

Plot 9: Video Confusion Matrix

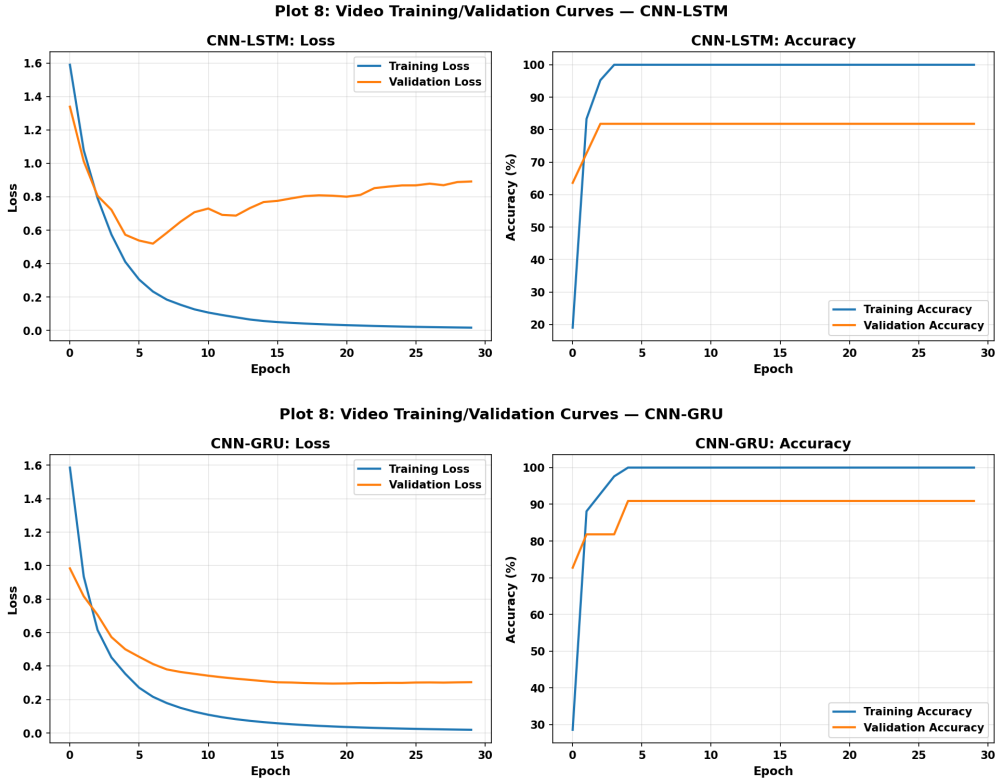
Analyze class-level recognition and temporal confusions.

Interpretation (Observed Result)

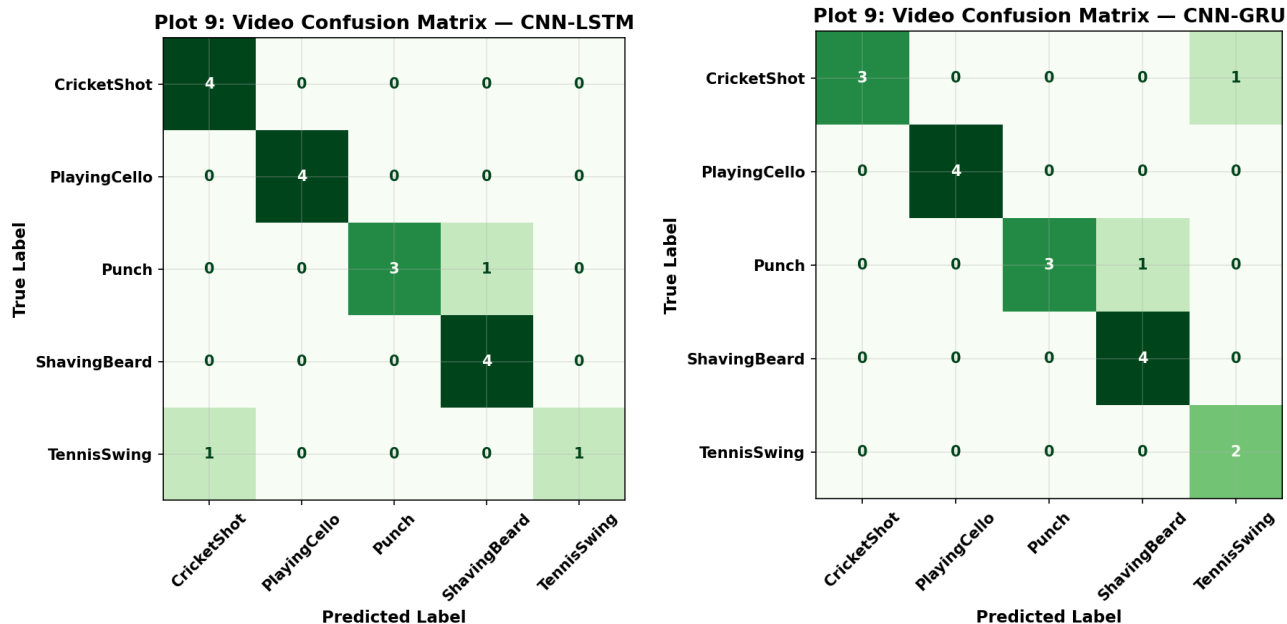
A 5-class UCF101 subset (CricketShot, PlayingCello, Punch, ShavingBeard, TennisSwing; 15 videos per class, 71 usable after frame-length filtering) was used. The frozen MobileNetV2 feature extractor produced a feature dimension of $D = 1280$, giving a feature tensor of shape $(71, 10, 1280)$ supplied to the recurrent network, exactly matching the $X \in \mathbb{R}^{B \times 10 \times D}$ form required in Section 19.



Plot 7 shows the 10 uniformly sampled, resized (224×224) frames for one representative video, showing the spatial input the CNN feature extractor receives per time step.



Plot 8 (CNN-LSTM and CNN-GRU training/validation curves): Both models learn quickly during the first few epochs. CNN-GRU shows more stable validation performance, while CNN-LSTM shows a slight increase in validation loss after the initial improvement, indicating some overfitting. Overall, CNN-GRU achieves better validation accuracy.



Plot 9 (Video confusion matrices): Both CNN-LSTM and CNN-GRU classify most videos correctly, as shown by the high values along the diagonal. A few errors occur between CricketShot and TennisSwing, and between Punch and ShavingBeard. Overall, CNN-GRU shows slightly fewer misclassifications and gives better classification performance.

Section 19 example (observed, from the trained model):

[CNN-LSTM]

Predicted class : ShavingBeard

Actual class : ShavingBeard

Confidence : 0.97

[CNN-GRU]

Predicted class : ShavingBeard

Actual class : ShavingBeard

Confidence : 0.96

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

Observed Dataset

20,000 synthetic 4-digit sequences (digits 0–9) were generated and split into 14,400 training, 1,600 validation and 4,000 test examples, e.g. $[6, 3, 7, 4] \rightarrow [4, 7, 3, 6]$.

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[0, 3, 9, 1]	[1, 9, 3, 0]
2	[6, 8, 6, 9]	[9, 6, 8, 6]
3	[2, 2, 6, 0]	[0, 6, 2, 2]
4	[5, 7, 7, 6]	[6, 7, 7, 5]
5	[2, 0, 7, 1]	[1, 7, 0, 2]

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

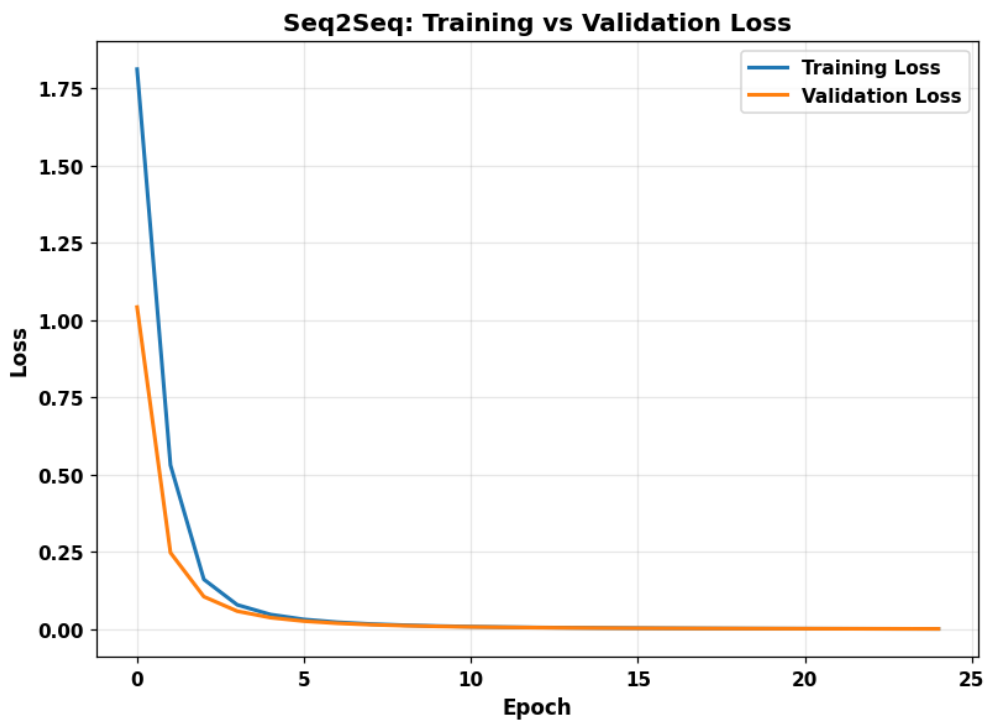
Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Interpretation (Observed Result)



Metric	Value
Token Accuracy	100.00%
Sequence Accuracy	100.00%
Final Training Loss	0.0008
Final Validation Loss	0.0008

The encoder-decoder LSTM achieves 100% token and sequence accuracy after 25 epochs because the fixed 4-digit reversal task is simple and fully learnable. Sequence accuracy is stricter than token accuracy since all positions must be correct. For longer or more difficult sequences, sequence accuracy is usually lower than token accuracy.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	80.44%	81.24%	80.44%	80.42%	1,974
LSTM	94.00%	94.04%	94.00%	93.99%	6,006
GRU	96.67%	96.67%	96.67%	96.67%	4,758
CNN-LSTM	88.89%	92.00%	85.00%	86.03%	168,677
CNN-GRU	88.89%	89.33%	90.00%	88.06%	126,757

A separate table must be provided for the sequence-to-sequence task.

Task	Token Accuracy	Sequence Accuracy	Training Loss	Validation Loss
Sequence Reversal (Seq2Seq)	100.00%	100.00%	0.0008	0.0008

26. Source Code

GitHub Repository

27. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

Reference to previous sections

Each mandatory inference above is addressed immediately after its corresponding plot/table in this report.

28. Discussion Questions

1. What is a recurrent neural network?
Answer: A neural network with feedback connections. It passes a hidden state across time steps so it can handle variable-length sequences using the same shared weights.
2. What information is represented by the hidden state?
Answer: It acts as a running memory buffer, keeping a compressed summary of past inputs to help make predictions at the current step.
3. Why is sequence order important?
Answer: Because the hidden state at step t depends directly on step $t-1$. Shuffling the order ruins the temporal context and destroys the meaning of time-series signals.
4. Explain the difference between a feed-forward network and an RNN.
Answer: A feed-forward net processes fixed inputs in a single forward pass without memory. An RNN loops activations back into itself to process sequential data over time.
5. Explain Backpropagation Through Time.
Answer: You unroll the network over all time steps into a deep computational graph, run backpropagation backward through time and sum the gradients for the shared weights.
6. Why can gradients vanish during BPTT?
Answer: During backprop, gradients pass through repeated matrix multiplications and tanh derivatives. If weights are below 1, the gradient shrinks exponentially and drops to zero before reaching early steps.
7. Why can gradients explode during BPTT?
Answer: If recurrent weights are greater than 1, repeated matrix multiplication causes gradients to grow exponentially, leading to numerical overflow and NaN values.
8. What are long-term dependencies?
Answer: When an output at the current step depends on an input that appeared many steps earlier in the sequence.
9. What is the role of the LSTM cell state?
Answer: It provides a linear path where memory is added or removed via gates rather than multiplied, letting gradients flow across long sequences without dying out.
10. What are the forget, input and output gates in LSTM?
Answer: The forget gate drops useless old information, the input gate adds new candidate info to the cell, and the output gate controls what part of the cell state gets passed to the hidden state.
11. What are the reset and update gates in GRU?
Answer: The reset gate controls how much past hidden state goes into the new candidate, while the update gate balances keeping the old state versus taking the new one.
12. Compare LSTM and GRU.
Answer: LSTM uses three gates and two separate states (cell and hidden), while GRU combines them into one state with just two gates. GRU trains faster and has fewer weights, while LSTM can model slightly more complex long-term patterns.
13. Why does GRU generally use a simpler gating mechanism than LSTM?

- Answer:** To cut computational cost and memory. Removing one gate and merging the states keeps the gradient highway intact while requiring fewer matrix operations.
14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
Answer: To ensure the comparison is fair. If data splits, learning rates, or optimizers differ, you cannot tell if accuracy gains came from the architecture or just better tuning.
 15. What does the confusion matrix reveal that accuracy alone does not?
Answer: It shows class-by-class errors. Accuracy hides when a model constantly mixes up specific pairs of classes, like sitting versus standing.
 16. Why should macro F1-score be reported for multi-class classification?
Answer: It averages the F1-scores of all classes equally, preventing majority classes from hiding terrible performance on small or hard classes.
 17. What is the significance of sequence length?
Answer: It sets how much context the model sees. Too short cuts off full movements like a step, while too long wastes memory and causes vanilla RNNs to suffer vanishing gradients.
 18. Why can a CNN be used as a feature extractor for video?
Answer: Pretrained CNNs already know general spatial features like edges, textures, and shapes from ImageNet, so they turn video frames into good feature vectors without extra training.
 19. What type of information is extracted by the CNN?
Answer: Static 2D spatial features within a single frame, like object shapes and scene layouts, with no concept of motion.
 20. What type of information is learned by the recurrent network?
Answer: Temporal dynamics across frames, tracking how spatial features change over time to identify actions and motion.
 21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
Answer: Because recurrent layers require sequential input shaped (B, T, D) to step through frames in chronological order.
 22. Explain the encoder and decoder in sequence-to-sequence learning.
Answer: The encoder compresses an input sequence into a single context vector, and the decoder reads that vector to generate the output sequence one step at a time.
 23. What is teacher forcing?
Answer: A training method where the decoder gets the true previous target as input instead of its own prediction, which keeps training stable and fast.
 24. Differentiate token accuracy and sequence accuracy.
Answer: Token accuracy is the percentage of individual correct outputs, while sequence accuracy requires the entire output string to be 100% correct.
 25. Why must the test set remain untouched during model selection?
Answer: Tuning hyperparameters or stopping training based on test results causes data leakage, giving an overly optimistic score that will not hold up on real-world data.

29. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
Observed: SimpleRNN improved sharply from 16 to 64 units (F1 rose from 70.83% to 91.71%) because more width helped offset its weak memory. In contrast, LSTM and GRU saturated around 16 units and saw minimal gains with larger layers.
2. Compare GRU and LSTM using the same number of recurrent units.
Observed: At 32 units, GRU beat LSTM on macro-F1 (96.67% vs 93.99%) while using about 21% fewer parameters (4,758 vs 6,006), making it clearly more parameter-efficient.
3. Add a second recurrent layer and investigate whether performance improves.
Observed: A second recurrent layer helped SimpleRNN (80.42% to 84.48%) and LSTM (93.99% to 95.77%), but hurt GRU (96.67% down to 93.93%) because the extra parameters overfit the dataset.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
Observed: Bidirectional LSTM gave a tiny boost in macro-F1 (94.20% vs 93.99%), but doubled the parameter count (11,894 vs 6,006) and increased training runtime by about 34%.
5. Change the sequence length and study the effect on computational cost.
Observed: Training time grew with sequence length for all three models. SimpleRNN was surprisingly the slowest per epoch, likely due to unoptimized GPU kernels rather than actual FLOP requirements.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.
Observed: Both models reached 88.89% test accuracy on frozen MobileNetV2 features, but GRU had a slightly higher macro-F1 (88.06% vs 86.03%), matching the trend from the sensor experiments.
7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

Observed: Mapping 4-digit inputs to 2-digit outputs reached 100% validation accuracy, confirming that the encoder-decoder context vector easily handles mismatched input and output lengths.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>